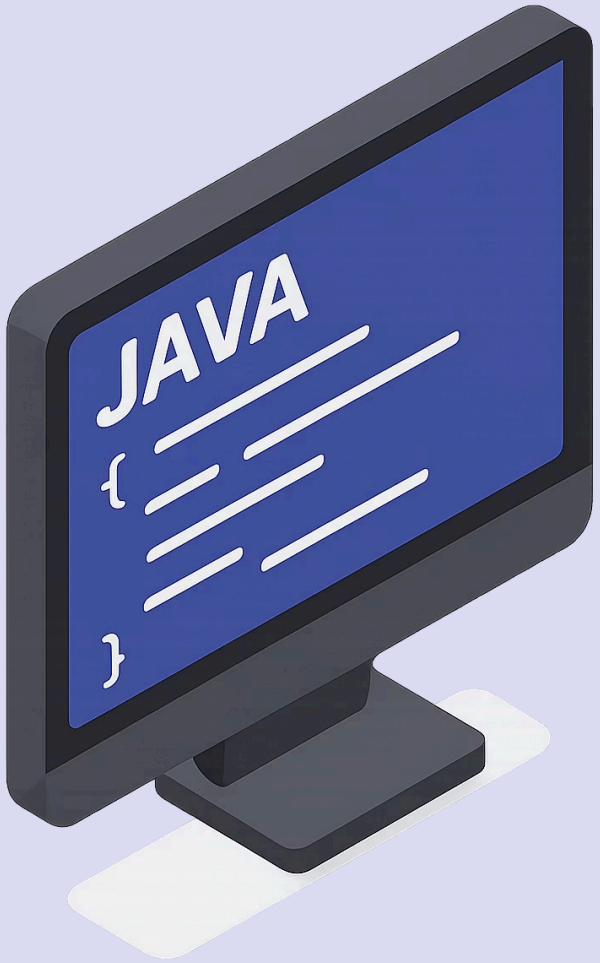
Mastering Wildcards in Java Generics

An advanced guide to wildcard usage with practical code examples



Made with GAMMA

What Are Wildcards?

Wildcards in Java generics represent unknown types using the `?` symbol. They provide flexibility when working with generic collections whilst maintaining type safety.

Think of wildcards as a way to say "I don't know exactly what type this is, but I know something about its relationship to other types."

Made with GAMMA

Three Types of Wildcards

Unbounded Wildcard

`List<?>` - Any type accepted

Upper-Bounded Wildcard

`List<? extends Number>` - Type or its subtypes

Lower-Bounded Wildcard

`List<? super Integer>` - Type or its supertypes

Made with GAMMA

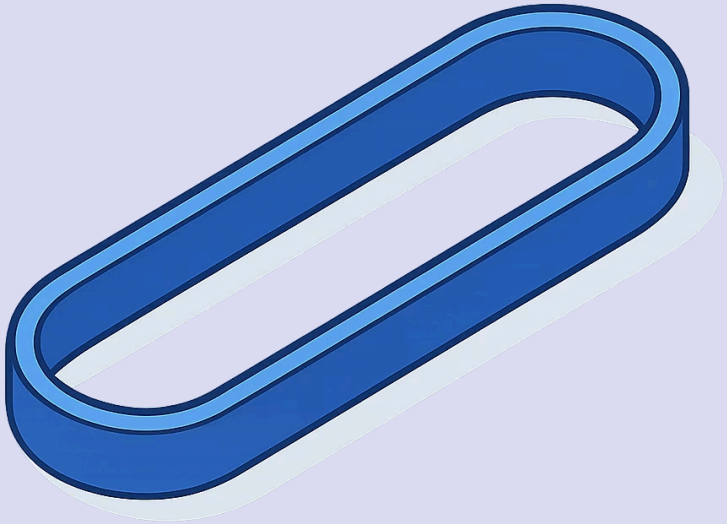
Unbounded Wildcards: The Flexible Choice

When to Use

When your method works the same way regardless of the type parameter, or when the type information is irrelevant to your operation.

Limitation

You cannot add elements to the collection (except `null`), as the compiler doesn't know what type is allowed.



Made with GAMMA

Unbounded Wildcard Example

Method Signature

```
List<?> list
```

Accepts a list of any type

Implementation

```
public static void printListSize(
    List<?> list) {
  System.out.println(
    "Size: " + list.size());
}

public static boolean isEmpty(
    List<?> list) {
  return list.isEmpty();
}
```

Notice how these methods only read information from the list—they don't need to know the specific type.

Made with GAMMA

Upper-Bounded Wildcards: The Restrictive Gatekeeper

Upper-bounded wildcards using `extends` restrict the unknown type to be a specific class or interface, or any of its subclasses.

Number

Base restriction

Integer

Subtype allowed

Double

Subtype allowed

Made with GAMMA

Upper-Bounded Wildcard Example

Method Definition

```
public static double sum(
    List<? extends Number> numbers) {
  double total = 0.0;
  for (Number num : numbers) {
    total += num.doubleValue();
  }
  return total;
}
```

Usage Examples

```
List<Integer> integers =
 Arrays.asList(1, 2, 3);
List<Double> doubles =
 Arrays.asList(1.5, 2.5, 3.5);

sum(integers); // ✓ Works
sum(doubles); // ✓ Works
sum(Arrays.asList("1")); //    Compile error
```

Made with GAMMA

Lower-Bounded Wildcards: The Flexible Container

Key Concept

Lower-bounded wildcards using `super` ensure the unknown type is a specific class or interface, or any of its supertypes.

Common Use Case

Perfect for method parameters where you need to add elements to a collection.

Made with GAMMA

Lower-Bounded Wildcard Example

01

Method Setup

```
public static void addNumbers(
    List<? super Integer> list) {
  list.add(42);
  list.add(100);
}
```

02

Compatible Collections

```
List<Integer> integers =
    new ArrayList<>();
List<Number> numbers =
    new ArrayList<>();
List<Object> objects =
    new ArrayList<>();
```

03

Calling the Method

```
addNumbers(integers); // ✓ Integer accepts Integer
addNumbers(numbers); // ✓ Number accepts Integer
addNumbers(objects); // ✓ Object accepts Integer
```

Made with GAMMA

Wildcards in Practice: Key Takeaways

Use unbounded wildcards for read-only operations

When type information doesn't matter and you're only reading from collections

Apply upper bounds for type-safe reading

When you need to ensure elements are of a specific type or its subtypes

Choose lower bounds for writing operations

When you need to add elements and want maximum flexibility for callers

Made with GAMMA